

Ant Colony Optimization for the Job Shop Scheduling Problem

Updated Technical Report: ACO, HiGHS MILP Baseline, and Visual Comparison

Adam DeJans Jr.

June 2026

Repository: github.com/addejans/ACO-JSSP
Live demo: addejans.github.io/ACO-JSSP
ACO implementation: `src/aco_jssp.py`
MILP baseline: `src/milp_jssp.py`
Comparison runner: `src/compare_solvers.py`
Visualizer: `docs/index.html`

Abstract

This report updates the original ACO-JSSP project documentation to reflect the modern Python 3 solver, generated instances, browser-based Gantt visualizer, HiGHS mixed-integer linear programming baseline, comparison runner, test suite, and GitHub Pages deployment. The project demonstrates how Ant Colony Optimization can be used as a constructive metaheuristic for the Job Shop Scheduling Problem and how a solver-backed MILP formulation can measure optimality gaps on smaller instances. The current implementation separates problem parameters from algorithm parameters, supports generated instances, applies a remaining-work-aware transition heuristic, mixes exploration with exploitation, reinforces both iteration-best and global-best solutions, applies pair-swap local search, and compares heuristic schedules against exact or bounded exact results.

Contents

1	Introduction	2
2	Problem Definition	2
2.1	Job precedence	2
2.2	Machine capacity	2
2.3	Objective	2
3	Repository Organization	2
4	ACO Solver Design	3
4.1	Schedule decoding	3
4.2	Transition rule	3
5	Additional ACO Techniques Implemented	3
5.1	Problem versus algorithm parameters	3
5.2	Exploration and exploitation	3
5.3	Pheromone evaporation and elite reinforcement	4
5.4	Pair-swap local search	4
6	Exact MILP Baseline with HiGHS	4
6.1	MILP variables	4
6.2	Makespan constraints	4
6.3	Disjunctive machine constraints	4
6.4	Comparison runner	4
7	Browser Visualizer and UI Comparison	5
8	Testing and Validation	5
9	Limitations	5
10	Future Work	5
11	Conclusion	6

1 Introduction

The Job Shop Scheduling Problem (JSSP) is a canonical problem in operations research and combinatorial optimization. A set of jobs must be processed on a set of machines. Each job has a fixed route through the machines, and each operation has a processing time. A feasible schedule must respect job precedence and machine capacity.

This repository began as a 2017 research prototype applying Ant Colony Optimization (ACO) to a small job-shop instance. The original version was useful as a learning artifact but difficult to maintain: it used Python 2 syntax, relied on an old Plotly API, used heavy global state, included hard-coded plotting credentials, and did not include tests or a modern UI. The updated repository keeps the historical scripts in `legacy/`, but moves active work into a cleaner Python 3 implementation with tests, a static UI, an exact MILP baseline, and report documentation.

The live visualizer is available at:

addejans.github.io/ACO-JSSP

2 Problem Definition

Let $J = \{1, \dots, n\}$ be the set of jobs and $M = \{1, \dots, m\}$ be the set of machines. Each job j contains an ordered route of operations, $O_j = (O_{j,1}, \dots, O_{j,m})$. Each operation $O_{j,k}$ has processing time $p_{j,k} > 0$ and required machine $\mu_{j,k} \in M$. In generated instances, each job visits each machine exactly once in a random route order.

2.1 Job precedence

$$s_{j,k+1} \geq s_{j,k} + p_{j,k} \quad \forall j, k = 1, \dots, m-1.$$

2.2 Machine capacity

For any two operations requiring the same machine, processing intervals cannot overlap:

$$[s_{j,k}, s_{j,k} + p_{j,k}) \cap [s_{j',k'}, s_{j',k'} + p_{j',k'}) = \emptyset.$$

2.3 Objective

The repository minimizes makespan:

$$C_{\max} = \max_{j,k} (s_{j,k} + p_{j,k}).$$

3 Repository Organization

Path	Purpose
<code>src/aco_jssp.py</code>	Modern Python 3 ACO solver, generated instances, CLI, schedule decoder, local search, and JSON output.
<code>src/milp_jssp.py</code>	Exact disjunctive MILP baseline using SciPy and the HiGHS solver backend.

<code>src/compare_solvers.py</code>	Command-line comparison runner for ACO versus HiGHS MILP.
<code>tests/test_aco_jssp.py</code>	Unit tests for schedule validity, determinism, generated instances, and optional HiGHS smoke tests.
<code>docs/index.html</code>	Static browser UI, Gantt visualizer, and small-instance exact comparison panel.
<code>paper/</code>	Updated LaTeX report source and compiled PDF.
<code>legacy/</code>	Original 2017 prototype scripts retained for historical reference.

4 ACO Solver Design

The ACO solver constructs a job-order sequence. If there are n jobs and m machines, the sequence has length $n \times m$. Each time job j appears, the decoder schedules the next unscheduled operation of that job at the earliest feasible time.

4.1 Schedule decoding

The decoder tracks job readiness, machine readiness, and the next operation index for each job. When job j is selected, the next operation starts at

$$s = \max(\text{jobReady}_j, \text{machineReady}_{\mu_{j,k}}).$$

4.2 Transition rule

At each construction step, an ant chooses among jobs that still have unscheduled operations. A job's attractiveness depends on pheromone and a heuristic score:

$$P(j) = \frac{\tau_{i,j}^\alpha \eta_j^\beta}{\sum_{\ell \in F} \tau_{i,\ell}^\alpha \eta_\ell^\beta}.$$

The updated heuristic is remaining-work-aware:

$$\eta_j = \frac{1}{\max(\text{earliestFinish}_j + 0.15 \cdot \text{remainingWork}_j, 1)}.$$

5 Additional ACO Techniques Implemented

5.1 Problem versus algorithm parameters

The UI separates problem parameters from algorithm parameters. Problem parameters define the instance: jobs, machines, duration range, and instance seed. Algorithm parameters define the search: ants, iterations, local search budget, and algorithm seed.

5.2 Exploration and exploitation

The solver mixes probabilistic weighted choice with a small amount of greedy exploitation. With probability q_0 , an ant chooses the highest-weight feasible move; otherwise it samples from the weighted transition distribution.

5.3 Pheromone evaporation and elite reinforcement

After each iteration, pheromone evaporates:

$$\tau_{i,j} \leftarrow (1 - \rho)\tau_{i,j}.$$

The solver reinforces the iteration-best order and applies an elite deposit to the global-best order:

$$\tau_{i,j} \leftarrow \tau_{i,j} + \frac{Q}{C_{\max}}.$$

5.4 Pair-swap local search

After an ant constructs a full job-order sequence, the solver applies pair-swap local search. It swaps two positions, decodes the candidate, and keeps the move if makespan does not worsen.

6 Exact MILP Baseline with HiGHS

The repository now includes an exact mixed-integer linear programming baseline in `src/milp-jssp.py`. The baseline is not meant to replace ACO. It is a measuring stick. For small and moderate instances, HiGHS can prove optimality or return a solver status, objective value, runtime, and MIP gap.

6.1 MILP variables

The formulation uses start-time variables s_o , makespan variable $C_{\max}$, and binary ordering variables $y_{a,b}$ for each pair of operations sharing a machine.

6.2 Makespan constraints

For every operation o with processing time p_o :

$$C_{\max} \geq s_o + p_o.$$

6.3 Disjunctive machine constraints

For two operations a and b on the same machine, let $y_{a,b} = 1$ mean operation a precedes operation b . With horizon H , the disjunction is linearized as:

$$s_b - s_a - Hy_{a,b} \geq p_a - H,$$

$$s_a - s_b + Hy_{a,b} \geq p_b.$$

6.4 Comparison runner

After installing SciPy, ACO and HiGHS can be compared from the command line:

```
python -m pip install -r requirements-optional.txt
python -m src.compare_solvers --jobs 4 --machines 3 --ants 40 --iterations 120 --milp-
time-limit 10
```

The output includes ACO makespan, ACO runtime, HiGHS objective, HiGHS runtime, solver status, MIP gap, and ACO gap when HiGHS proves optimality.

7 Browser Visualizer and UI Comparison

The browser visualizer is a static HTML page under `docs/index.html`. It runs directly on GitHub Pages. Since SciPy/HiGHS cannot run inside a static browser page, the UI includes a capped exact branch-and-bound comparison for small instances. It shows ACO makespan, exact makespan when proven, runtime, search nodes, and ACO gap.

Control group	Controls / outputs
Problem parameters	Jobs, machines, minimum duration, maximum duration, instance seed.
Algorithm parameters	Ants, iterations, local search budget, algorithm seed, exact-search time limit.
Comparison outputs	ACO runtime, exact runtime, ACO makespan, exact makespan, exact status, exact nodes, and ACO gap.
Visualization outputs	Gantt chart, job legend, and schedule JSON.

8 Testing and Validation

The tests focus on correctness properties that should hold regardless of stochastic quality: complete schedules, job route precedence, machine non-overlap, reproducible generated instances, deterministic ACO for fixed seeds, and optional HiGHS MILP smoke tests when SciPy is installed.

```
python -m unittest discover -s tests -v
```

9 Limitations

The HiGHS MILP baseline is valuable for small instances and quality comparisons, but exact formulations can become expensive as the number of jobs and machines grows. The browser exact search is deliberately capped to keep GitHub Pages responsive. The current ACO model learns pheromone on transitions between job selections rather than richer operation-level or machine-level sequencing decisions.

10 Future Work

1. Add standard benchmark instances such as FT06, LA, ORB, and ABZ families.
2. Extend the HiGHS comparison runner with benchmark tables and solver diagnostics.
3. Add a CP-SAT baseline for comparison against the MILP formulation.
4. Add critical-path local search neighborhoods.
5. Add machine-specific pheromone matrices.
6. Add dispatching-rule hybrids such as SPT, LPT, slack, and bottleneck-aware rules.
7. Add runtime and convergence charts.
8. Export schedules to CSV and JSON from the browser UI.

11 Conclusion

The updated ACO-JSSP repository is now a coherent optimization demonstration. It includes a Python ACO solver, generated instances, an exact HiGHS MILP baseline, a solver comparison runner, a static browser UI, Gantt visualization, unit tests, GitHub Pages deployment, and a LaTeX-style technical report. The practical lesson is that metaheuristics become much more useful when paired with clean representations, validation, exact baselines, reproducibility, and visualization.

References

- [1] M. Dorigo, V. Maniezzo, and A. Coloni. “Ant System: Optimization by a Colony of Cooperating Agents.” *IEEE Transactions on Systems, Man, and Cybernetics*, 26(1), 1996.
- [2] M. Dorigo and T. Stuetzle. *Ant Colony Optimization*. MIT Press, 2004.
- [3] M. R. Garey, D. S. Johnson, and R. Sethi. “The Complexity of Flowshop and Jobshop Scheduling.” *Mathematics of Operations Research*, 1(2), 1976.
- [4] J. Blazewicz, W. Domschke, and E. Pesch. “The Job Shop Scheduling Problem: Conventional and New Solution Techniques.” *European Journal of Operational Research*, 93(1), 1996.